PROJET 3 : Requêtez une base de données avec SQL

Document Technique

Exploration des données

1) Les types de données

Lors de la récupération des types de données il était nécessaire d'identifier les types de données et la nature des données présentes dans la base

Chaque colonne doit être définie avec un type adapté (ex : INT, TEXT, DATE etc..) cette étape garantit que les opérations (calculs agrégations, jointures) sont possibles et cohérentes
La nature des données s'agit de comprendre ce que représente chaque colonne (contrat, région, département etc..) cela permet de savoir comment les utiliser dans les requêtes et d'éviter les erreurs logiques.

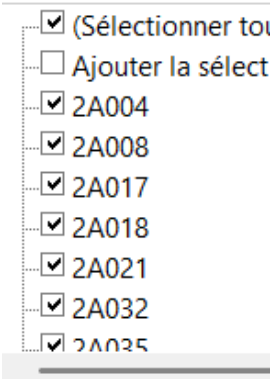
La nature de chaque colonne de notre tableau ci-dessous

Code_dep_commune : texte Reg_code : Nombre entier Aca_nom : texte Dep_nom : texte	Code_dep_co reg_code reg_nom aca_nom dep_nom 1001 84 Auvergne-Rhô Lyon Ain 1002 84 Auvergne-Rhô Lyon Ain 1003 84 Auvergne-Rhô Lyon Ain 1004 84 Auvergne-Rhô Lyon Ain 1005 84 Auvergne-Rhô Lyon Ain 1006 84 Auvergne-Rhô Lyon Ain 1007 84 Auvergne-Rhô Lyon Ain 1008 84 Auvergne-Rhô Lyon Ain 1009 84 Auvergne-Rhô Lyon Ain

2) Le choix des natures :

Pour bien identifier la nature de la colonne, il faut parfois bien chercher et pas s'en tenir à regarder les premières ligne. Les bases de données peuvent être très volumineuse et dans ce cas essayer

d'autre fonction comme celle de FILTRE pour voir s'il ne se cache pas du texte au milieu des nombres entier ou l'inverse.
Exemple pour notre cas : dans la colonne Code_dep_commune

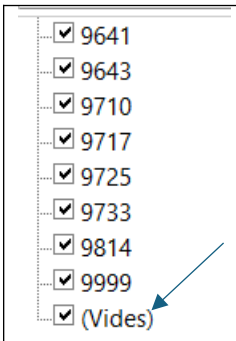


À première vue, on aurait pu penser que la nature de cette colonne correspondait à des nombres entiers. Cependant, ces valeurs numériques sont en réalité stockées sous forme de texte. Ainsi, même si elles ressemblent à des nombres, elles sont interprétées par MySQL comme des chaînes de caractères.

En revanche, j'ai constaté qu'il existe des nombres qui peuvent être stockés au format texte au lieu d'être reconnu comme de véritable valeurs numérique les causes :

- **Alignement par défaut** : dans Excel les nombres au format numérique sont généralement alignés à droite, tandis que les valeurs au format texte sont alignées à gauche, ainsi le nombre affiché à gauche peux être traité comme du texte
- **Absence de lettres cachées :** si la cellule ne contient aucune lettre ou caractère non numérique, il est possible de convertir ces valeurs en format nombre
- **Conséquence :** Tant que les données restent en texte, Excel et SQL ne peuvent pas effectuer correctement des calculs, il faut donc convertir les colonnes concernées en format nombre.

Attention, d'autre colonnes peuvent cacher des valeurs vides. Il faudra également adapter les bonnes contraintes d'integrité

	Contrainte : • MySQL stocke alors une valeur spéciale NULL (qui n'est pas un 0 ni une chaine vide) • NULL : autorise les valeurs vides (absence de donnée)

A contrario : Le NOT NULL oblige à ce que chaque ligne ait une valeur dans cette colonne, si le fichier CSV contient une case vide, l'import échoue.

La compréhension des données

Comprendre les données est la première étape avant de les analyser ou de les exploiter. Il est toujours utile de comprendre le rôle des données

Le fichier « Région » : sert à contextualiser les données selon la localisation, facilite la segmentation et la comparaison entre zones géographiques

Code_dep_co	reg_code	reg_nom	aca_nom	dep_nom	com_nom_ma	dep_code	dep_nom_num
1001	84	Auvergne-Rhô	Lyon	Ain	L ABERGEMEN	1	Ain (01)

Le fichier « Contrat » : Contient les informations sur les engagements, les durées, les types de services ou prix de cotisation etc.

Voie	Code_dep_co	Code_postal	Surface	Type_local	Occupation	Type_contrat	Formule	Valeur_declar	Prix_cotisation_mensuel
CENTRALE	1350	1370	50	Appartement	Locataire	Residence pri	Integral	0-25000	25

Ces 2 fichiers, une fois croisés offrent une vision complète : on peut relier les dimensions géographiques à la dimension contractuelle (quoi, comment, combien). Cela permet de transformer des données brutes en indicateurs utiles pour la prise de décision.

Le dictionnaire des données

Lorsque les fichiers ont été téléchargés et que les données sont comprises, la prochaine étape est remplir le dictionnaire des données ont doit y mettre le nom des colonnes, type de donné, la taille s'y nécessaire, la clé si elle existe, et une description de chaque colonne

	Nom des colonnes	Type de données	Taille	Clé	Description
CONTRAT.CSV	Contrat_ID	INT		Clé primaire	Id unique pour les contrats
	No_voie	VARCHAR	50		Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	CHAR	1		Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	VARCHAR	20		Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR	100		Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	CHAR	6	Clé secondaire	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	INT			Code postal pour l'adresse du logement assuré
	Surface	INT			surface du bien
	Type_local	VARCHAR	20		typologie du logement (appartement ou maison)
	Occupation	VARCHAR	20		qualité de l'occupant (propriétaire ou locataire)
	Type_contrat	VARCHAR	50		type de contrat (residence principale, secondaire, location)
	Formule	VARCHAR	20		type de formule (classique ou integrale)
	Valeur_declaréé_biens	VARCHAR	20		Tranche de la valeur du bien
	Prix_cotisation_mensuel	INT			prix cotisation mensuel
REGION.CSV	Code_dep_code_commune	VARCHAR	10	Clé primaire	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INT			code de la région
	reg_nom	VARCHAR	50		nom de la région
	aca_nom	VARCHAR	50		nom de l'academie
	dep_nom	VARCHAR	50		nom du département
	com_nom_maj_court	VARCHAR	50		nom de la commune
	dep_code	CHAR	3		numéro du département
	dep_nom_num	VARCHAR	50		concatenation du nom du département et du numéro du département

Quelques explications :

Les types de données peuvent être selon la colonne soit :

- INTEGER : sert à stocker des nombres entiers (positifs ou négatifs), utilisé pour des identifiants, codes, numéro de voie
- CHAR (n) : stock une chaîne de caractères de longueur fixe, si on définit CHAR (10) et que l'on insère ABC, MYSQL complète avec des espaces pour atteindre 10 caractères. Rapide pour des codes courts et toujours de même taille. Les inconvénients : gaspille de l'espace si les valeurs sont plus courtes. Comme pour B_T_Q où l'on a un seul caractère
- VARCHAR(n) : Stocke une chaîne de caractères de longueur variable, jusque n. VARCHAR (100) peut contenir de 0 à 100 caractères. Avantage : flexible, ne gaspille pas d'espace, il faut juste choisir une taille adaptée (ni trop petite, ni grande)

Les types de données définissent la nature des valeurs stockées dans une table SQL.

Les CLES : primaire et étrangère

La clé primaire est un champ ou ensemble de champs qui

- Identifie de manière unique chaque ligne d'une table
- Ne peut pas être NULL
- Sert de référence principale pour retrouver une ligne

Dans la table Contrat, la colonne Contrat_ID est la clé primaire car chaque contrat a un code unique.

La clé étrangère (FOREIGN KEY) est un champ dans une table qui fait référence à la clé primaire d'une autre table son rôle

- Crée un lien logique entre 2 tables
- Assurer l'intégralité référentielle : on ne peut pas insérer une valeur qui n'existe pas dans une table de référence.

Dans la table contrat, la colonne Code_Dep_Code_Commune est une clé étrangère qui pointe vers la table région

Ces clés ensemble permettent de construire une base relationnelle solide, où les données sont fiables et bien organisées.

La description dans le dictionnaire des données

La description est indispensable

- Pour la communication entre équipes : les développeurs, analystes et autres métiers n'ont pas toujours la même lecture des colonnes, elle évite les ambiguïtés.
- Pour une documentation durable : quand on revient sur le projet 6 mois plus tard ou qu'un collègue reprend le travail, la description permet de comprendre rapidement la logique des données.
- Contrôler la qualité : elle aide à vérifier que les données importées correspondent bien à ce qui est attendu (exemple : un champ « Surface » doit être en M² et pas en nombre de pièces)

La colonne description apporte une compréhension métier des champs, au-delà de leurs types techniques. Elle permet de clarifier l'usage attendu, de facilité la communication entre équipe et de garantir la cohérence des données lors de l'importation et l'exploitation.

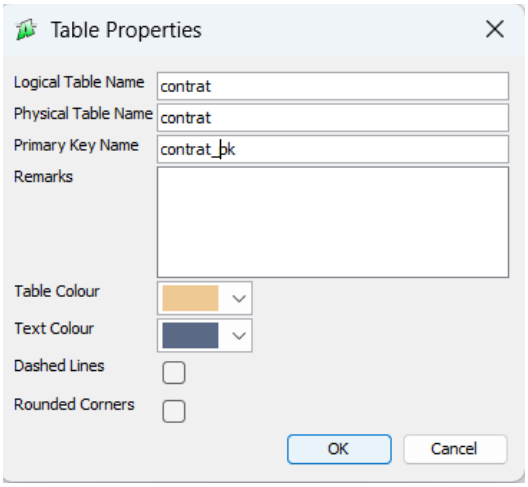
Le nom des types de données

Le nom du type de données peut varier en fonction de la SGBD que l'on utilise, dans notre cas nous allons utiliser la SGBD MySQL

Données	MySQL
Nombre entier	INTEGER
Nombre décimal	REAL
Texte longueur fixe	CHAR(n)
Texte longueur variable	VARCHAR(n)
Date	DATE
Heure	TIME
Date et heure	DATETIME

Donnée dans SQL ARCHITECTURE

Dès que l'étape de bien comprendre et remplir le dictionnaire de données est finie, il faut retranscrire toutes les informations sur SQL architecture pour générer un code SQL



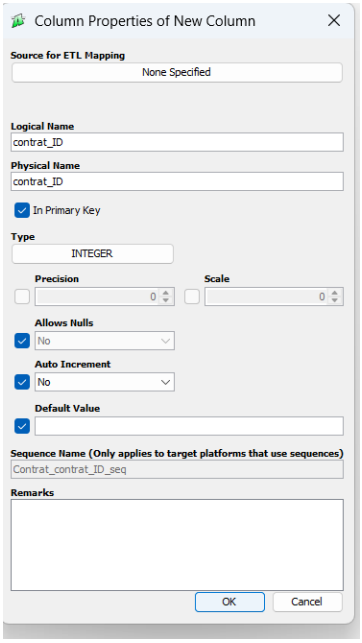
Etape 1 : Création des tables

Pour crée une table il faut cliquer sur l'icône à droite « New table »

Dessiner avec la croix un carré avec la souris et une fenêtre « table properties » s'ouvre. Ensuite indiqué les informations à la création de la table, pour les 2 premiers champs indiqué la même information.

La personnalisation est possible : couleur etc

Etape 2 : Création des colonnes



Dès que l'étape de la création de la table est finie, il faut passer à l'étape crée les colonnes. Il faut aller sur l'onglet à droite « insert column » et donner le nom de la colonne sur « Logical Name et Physical Name »

Nous allons cochet que c'est une clé primaire « In primary key », et préciser le type de donnée (confère dictionnaire de données) « INTEGER »

Si des données vides sont autorisées, il faut cocher « yes » dans « Allows Nulls » sinon cocher « No ». Les autres informations j'ai laissé par défaut

---

LA JOINTURE

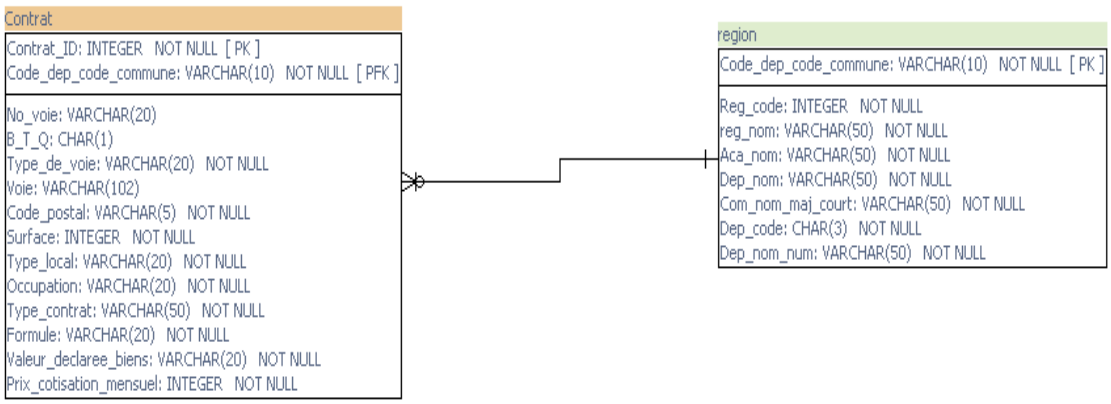
La jointure garantit que chaque contrat est lié à une région valide, nous allons dessiner les liens entre les tables pour mieux comprendre la structure. Dans le dictionnaire des données la clé primaire est « Contrat_ID » de la table « Contrat », et « Code_Dep_Code_Commune » est la clé primaire de la table « région ». Ces données seront tous en haut de l'architecture avec les initiales « PK »





Pour faire la jointure il faut aller dans l'interface à droite dans l'onglet « news identifying Relationship » et ensuite relier la clé primaire de la table « région » « Code_dep_code_commune » et finir par cliquer sur la clé étrangère de la table « contrat » « Code_dep_code_commune »

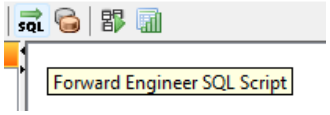
Nous pouvons voire que notre schéma architecture est bien réalisé



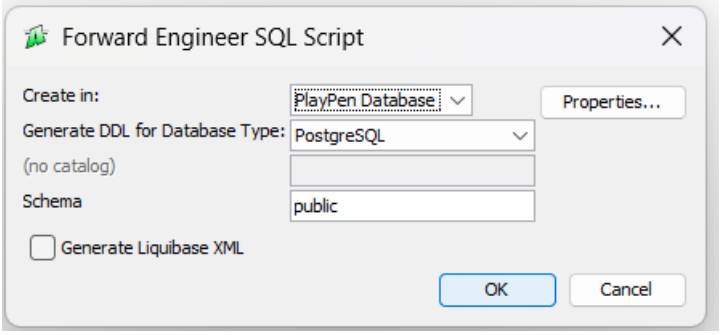
Générer le code SQL

Quand on s'est bien assuré que les jointures ont bien été réalisées, l'architecture est réalisée correctement, la dernière étape est de générer le code SQL

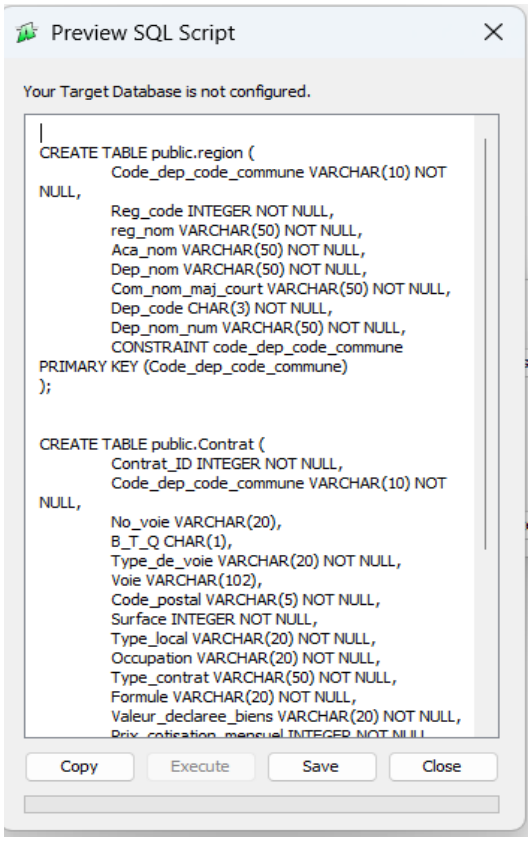
Pour cela, dans l'interface tous en haut il y a une flèche verte



Cliquer dessus et une fenêtre s'ouvre

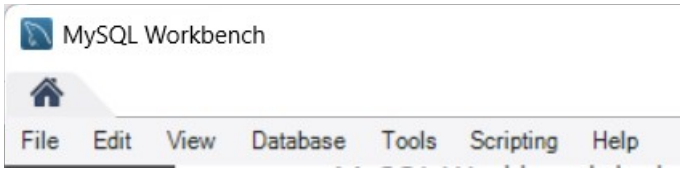


Je dois m’assurer que le « Generate DDL for Database type » est bien « PostgreSQL » et cliquer sur ok ensuite le code SQL se génère automatiquement il est aussi possible de le sauvegarder « copy »

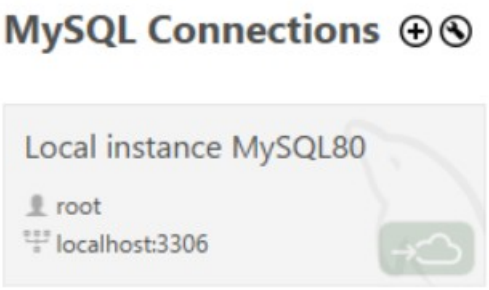


Téléchargement de la base de données

Nous allons télécharger la base de données avec lequel nous allons travailler, ici je vais utiliser la SGBD MYSQLWORKBENCH.



Je vais crée un mot de passe pour accéder directement à mon poste de travail

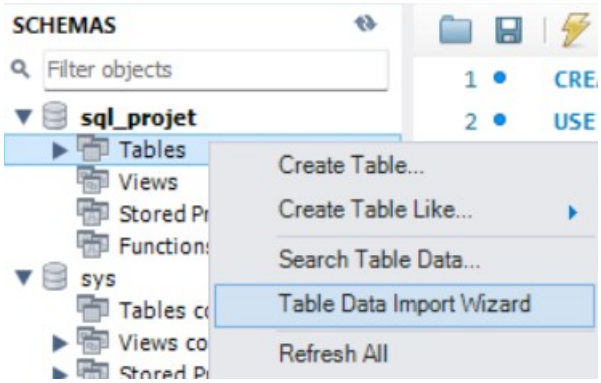


Création de la base de données :

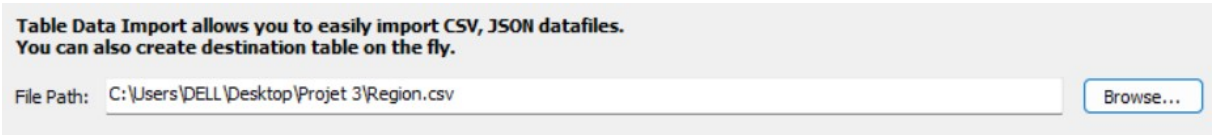
Tous d'abord, il faut crée notre base de données, ici je l'appellerai « Sql_projet ». Dans l'éditeur le code SQL que j'ai utilisé est celui-ci :

```
1 •  CREATE DATABASE SQL_projet;
2 •  USE SQL_projet;
```

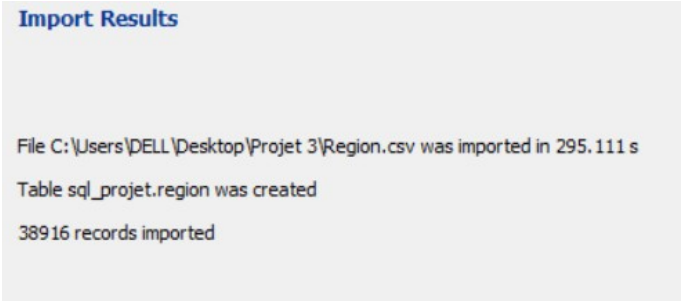
Une fois ma base de données crée, je vais pouvoir crée mes tables et importer mes données. Pour crée mes tables voici l'étape à suivre juste en dessous de la BDD sql_projet je clic droit sur « Tables » et je vais l'onglet « Table Data Import Wizard »



Il faut charger le fichier CSV, et préciser l'encodage qui à été déterminé au moment de l'analyse des données. Le fichier « région » est en UTF8



A la fin de l'import la procédure nous indique que le compte est bon par ex : pour le fichier « région »



Autre méthode : en faisant une requête SQL : SELECT COUNT (*) FROM region ; SELECT COUNT (*) FROM contrat ;

	COUNT(*)
▶	38916

	COUNT(*)
▶	30335

Autre manière de crée une table et faire l'import des données

Voici une autre manière de crée une table avec un code SQl

Code SQL : création d'une table	Code SQL : importation de la table

```sql
CREATE TABLE contrat (
    Contrat_ID INT PRIMARY KEY,
    No_voie VARCHAR(10) NULL,
    B_T_Q CHAR(1),
    Type_de_voie VARCHAR(20) NOT NULL,
    Voie VARCHAR(100) NULL,
    Code_dep_code_commune CHAR(6) NOT NULL,
    Code_postal VARCHAR(5) NOT NULL,
    Surface INT NOT NULL,
    Type_local VARCHAR(20) NOT NULL,
    Occupation VARCHAR(20) NOT NULL,
    Type_contrat VARCHAR(50) NOT NULL,
    Formule CHAR(9) NOT NULL,
    Valeur_declaréé_biens VARCHAR(20) NOT NULL,
    Prix_cotisation_mensuel INT NOT NULL
);
```

```sql
LOAD DATA INFILE "C:\Users\DELL\Desktop\Projet 3\Contrat.csv"
INTO TABLE contrat
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Les requêtes SQL

1) Les requêtes simples

Les requêtes SQL c'est un langage qui permet de dialoguer avec ma base, elles permettent d'interagir avec la base de données. Elles se déclinent en plusieurs catégories via la commande SELECT, FROM, des filtres via la commande WHERE, des classements via la commande ORDER BY enfin la limitation des données via la commande LIMIT

- Requête 1 : Lister les numéros de contrats (contrat_ID) avec leur surface pour le code postal 92100.

```
SELECT contrat_ID, Surface
FROM contrat1
WHERE code_postal =
92100 ;
```

Contrat_ID	Surface
123026	61
123027	31
123028	17
123030	78
123030	109
123033	87
123034	80

- Requête 2 : Lister le nom des régions de France

```
SELECT DISTINCT reg_nom
FROM region1 ;
```

reg_nom
Auvergne-RhÃ´ne-Alpes
Hauts-de-France
Provence-Alpes-CÃ´te d'Azur
Grand Est
Occitanie
Normandie
Nouvelle-Aquitaine

- Requête 3 : Combien existe-il de contrats sur les résidences principales ?

```
SELECT COUNT (contrat_ID) AS
nb_contrats_residences_principales
FROM contrat1
Where Type_local = 'Résidence
principale' ;
```

nb_contrats_residences_principales
25513

2) Les requêtes avec les agrégations et jointure

Les requêtes avec agrégations et jointure fonctionnent de la même manière que les requêtes simples. Par contre les calculs sont un peu plus poussé tel que les calculs de moyenne, somme en utilisant les fonctions d'agrégation suivante

AVG() : calcul de la moyenne
SUM() : calcul de la somme
COUNT() : compte le nombre de valeur ou de ligne

Min() : minimum/ Max() : maximum
Group by : donne le nombre par (colonne choisit)
ORDER BY() :Trie par défaut ou ordre croissant/ DESC : ordre décroissant
LEFT JOIN() : relie 2 tables et garde toute les lignes de la table de gauche, même si pas de correspondance dans la table de droite

Requête 4 : Quels sont les 5 contrats qui ont les surfaces les plus élevées ?

```
SELECT Contrat_ID, Surface
FROM    contrat1
ORDER BY Surface DESC
LIMIT 5 ;
```

Contrat_ID	Surface
104211	815
105463	742
130878	595
100822	570
109872	559

Requête 5 : Quel est le prix moyen de la cotisation mensuelle ?

```
SELECT
AVG(Prix_cotisation_mensuel) AS
Prix_moyen_cotisation
FROM contrat ;
```

Prix_moyen_cotisation
19.3287

Requête 6 : Quel est le nombre de contrats pour chaque catégorie de prix de la valeur déclarée des biens ?

```
SELECT valeur_declaree_biens
COUNT(Contrat_ID) AS
nb_contrat
FROM Contrat
GROUP BY
Valeur_declaree_bien
ORDER BY nb_contrats DESC ;
```

Valeur_declaréé_biens	nb_contrats
0-25000	22720
25000-50000	6815
50000-100000	696
100000+	104

Requête 7 : Quel est le nombre de formules “intégral” sur la région Pays de la Loire ?

```
SELECT COUNT (contrat_ID) AS
nb_contrats_integral_PaysLoire
 FROM contrat c
LEFT JOIN region r
ON c.Code_dep_code_commune
= r.Code_dep_code_commune
WHERE Formule = 'intégral'
AND reg_nom = 'Pays de la Loire';
```

nb_contrats_integral_PaysLoire
589

- Requête 11 : Liste des communes ayant eu au moins 150 contrats. (20 résultats)

```
SELECT r.com_nom_maj_court as commune,
COUNT(c.contrat_ID) as nb_contrats
FROM contrat c
JOIN region r
ON c.code_dep_code_commune = r. code_dep_code_commune
GROUP BY r.com_nom_maj_court
HAVING COUNT(c.contrat_ID)>=150
ORDER BY nb_contrats DESC ;
```

commune	nb_contrats
PARIS 18	515
PARIS 17	468
PARIS 15	407
PARIS 16	394
NICE	386
PARIS 11	381
BORDEAUX	302
PARIS 20	302
NANTES	291
PARIS 19	266

Requête 12 : Quel est le nombre de contrats pour chaque région ? (16 résultats)

```
SELECT r.reg_nom as region,
COUNT(c.contrat_ID) as nb_contrats
FROM contrat c
JOIN region r

ON c.code_dep_code_commune = r.code_dep_code_commune
GROUP BY r.reg_nom
ORDER BY nb_contrats DESC ;
```

region	nb_contrats
Ile-de-France	14177
Provence-Alpes-CÃ´te d'Azur	3279
Auvergne-RhÃ´ne-Alpes	3042
Nouvelle-Aquitaine	2038
Occitanie	1609
Pays de la Loire	1196
Hauts-de-France	1189
Bretagne	947
Normandie	824
Grand Est	769
Centre-Val de Loire	598
Bourgogne-Franche-ComtÃ©	293